

	SR UNIVERSITY	
	Campus Warangal	Batch: 66
	Program: II - B. Tech (CS& AI)	Name: Shaishab Saha
	Faculty's: Dr. Mathivanan	Reg. No: 2503A51688
	Department: Computer Science and AI	Semester: I
AI Assisted Coding-Assignment:12 Date:21-09-2026		
General Instructions • Run the original program before making changes. • Identify bugs systematically before asking AI for a complete solution. • Use AI to explain causes and suggest fixes. • Verify every AI suggestion by testing it yourself. • Record the bug, cause, fix and test result. • Do not blindly copy AI-generated code.		

Q1 – AI-Assisted Bubble Sort Analysis

Problem / Starter Code:

```
numbers = [64, 34, 25, 12, 22, 11, 90]
```

```
# TODO:
```

```
# Implement Bubble Sort in ascending order.
```

```
# Count comparisons and swaps.
```

```
# Display the sorted list.
```

Student Tasks:

- Implement Bubble Sort.
- Use AI to explain one complete pass of Bubble Sort.
- Count comparisons and swaps.
- Test at least three input lists.
- Ask AI about an early-exit optimisation.
- Compare the original and improved versions.

The screenshot shows the Visual Studio Code interface. The Explorer panel on the left shows a folder named 'AI CODING' containing files Q1.py, Q2.py, Q3.py, Q4.py, and Q5.py. The main editor window displays the code for Q1.py, which implements a bubble sort algorithm. The code includes a function 'bubble_sort' that takes a list of numbers, sorts them, and returns the sorted list along with the number of comparisons and swaps. The main part of the code initializes a list of numbers [64, 34, 25, 12, 22, 11, 90] and prints the sorted list, comparisons, and swaps. The output panel at the bottom shows the execution results: 'Sorted: [11, 12, 22, 25, 34, 64, 90]', 'Comparisons: 21', and 'Swaps: 14'.

```
Q1.py
1  def bubble_sort(numbers):
2      arr = numbers.copy()
3      comparisons = swaps = 0
4
5      for i in range(len(arr) - 1):
6          swapped = False
7          for j in range(len(arr) - 1 - i):
8              comparisons += 1
9              if arr[j] > arr[j + 1]:
10                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
11                 swaps += 1
12                 swapped = True
13          if not swapped:
14              break
15      return arr, comparisons, swaps
16
17 numbers = [64, 34, 25, 12, 22, 11, 90]
18 result, comparisons, swaps = bubble_sort(numbers)
19 print("Sorted:", result)
20 print("Comparisons:", comparisons)

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS C:\Users\Student\Desktop\AI CODING> python Q1.py
Sorted: [11, 12, 22, 25, 34, 64, 90]
Comparisons: 21
Swaps: 14
```

Output: Sorted = [11, 12, 22, 25, 34, 64, 90]; Comparisons = 21; Swaps = 14.

One complete pass:

64>34 → swap → [34,64,25,12,22,11,90]

64>25 → swap → [34,25,64,12,22,11,90]

64>12 → swap → [34,25,12,64,22,11,90]

64>22 → swap → [34,25,12,22,64,11,90]

64>11 → swap → [34,25,12,22,11,64,90]

64<90 → no swap → [34,25,12,22,11,64,90]. The largest element 90 is now at the end.

Three test cases:

[1,2,3,4,5] → [1,2,3,4,5], 4 comparisons, 0 swaps.

[5,4,3,2,1] → [1,2,3,4,5], 10 comparisons, 10 swaps.

The early-exit optimisation stops when a complete pass makes no swaps. This reduces unnecessary work for already sorted or nearly sorted data.

Complexity: average/worst case $O(n^2)$; best case with early exit $O(n)$; extra algorithmic space $O(1)$.

Q2 – Linear Search vs Binary Search

Problem / Starter Code:

```
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
```

```
target = 45
```

```
# TODO:
```

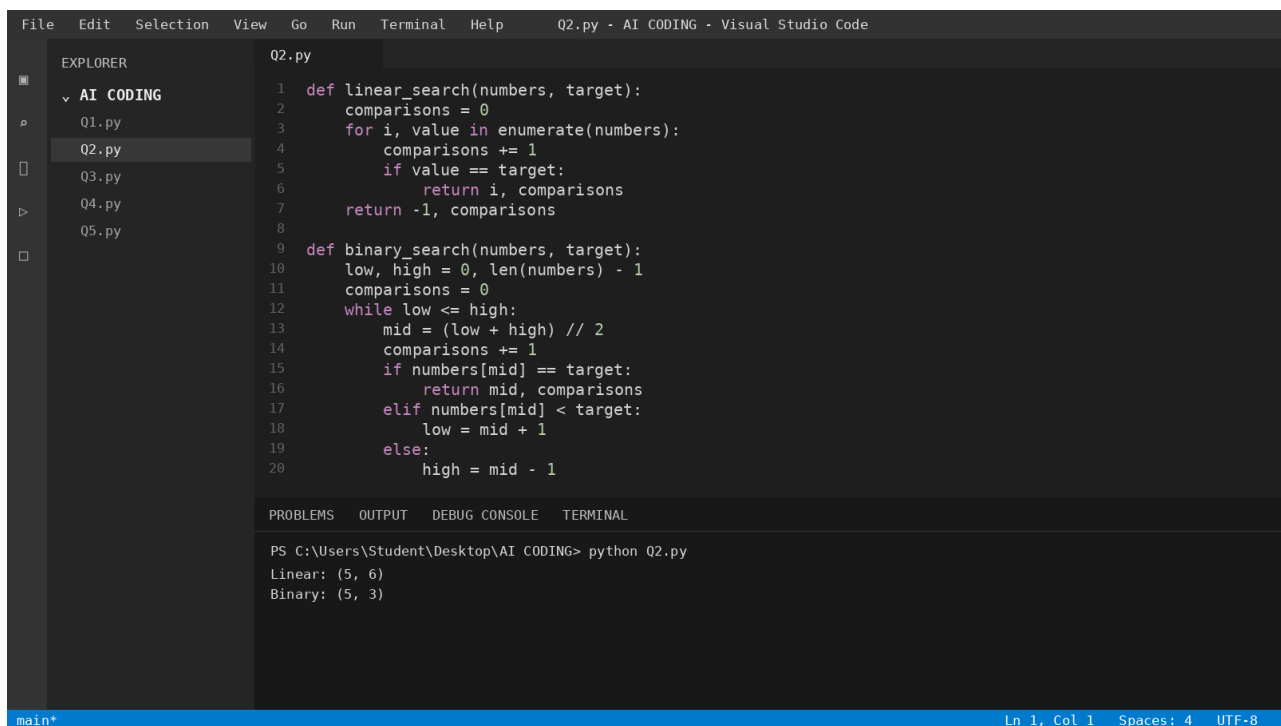
```
# Implement Linear Search.
```

```
# Implement Binary Search.
```

```
# Compare the number of comparisons.
```

Student Tasks:

- Implement Linear Search and Binary Search.
- Use AI to explain why Binary Search requires sorted data.
- Count comparisons for both algorithms.
- Test first, middle, last and absent elements.
- Compare the results and explain suitable use cases.
- Use AI to suggest a valid optimisation.



```
File Edit Selection View Go Run Terminal Help Q2.py - AI CODING - Visual Studio Code

EXPLORER
  AI CODING
    Q1.py
    Q2.py
    Q3.py
    Q4.py
    Q5.py

Q2.py
1 def linear_search(numbers, target):
2     comparisons = 0
3     for i, value in enumerate(numbers):
4         comparisons += 1
5         if value == target:
6             return i, comparisons
7     return -1, comparisons
8
9 def binary_search(numbers, target):
10     low, high = 0, len(numbers) - 1
11     comparisons = 0
12     while low <= high:
13         mid = (low + high) // 2
14         comparisons += 1
15         if numbers[mid] == target:
16             return mid, comparisons
17         elif numbers[mid] < target:
18             low = mid + 1
19         else:
20             high = mid - 1

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\Student\Desktop\AI CODING> python Q2.py
Linear: (5, 6)
Binary: (5, 3)

main* Ln 1, Col 1 Spaces: 4 UTF-8
```

For target 45: Linear Search finds it at index 5 after 6 comparisons. Binary Search finds it at index 5 after 3 comparisons.

Required tests:

First element 5 → Linear: 1 comparison; Binary: 3 comparisons; Found.

Middle element 31 → Linear: 5 comparisons; Binary: 1 comparison; Found.

Last element 89 → Linear: 9 comparisons; Binary: 4 comparisons; Found.

Absent element 50 → Linear: 9 comparisons; Binary: 3 comparisons; Not found.

Why Binary Search needs sorted data:

Binary Search compares the target with the middle element and discards one half of the list. This is safe only when the values are ordered; otherwise the discarded half may contain the target.

Optimisation: use Binary Search when the data is already sorted and there are many searches. Linear Search remains suitable for small or unsorted data.

Complexity: Linear Search $O(n)$ time and $O(1)$ extra space. Iterative Binary Search $O(\log n)$ time and $O(1)$ extra space.

Q3 – Compare Sorting Algorithms

Problem / Starter Code:

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

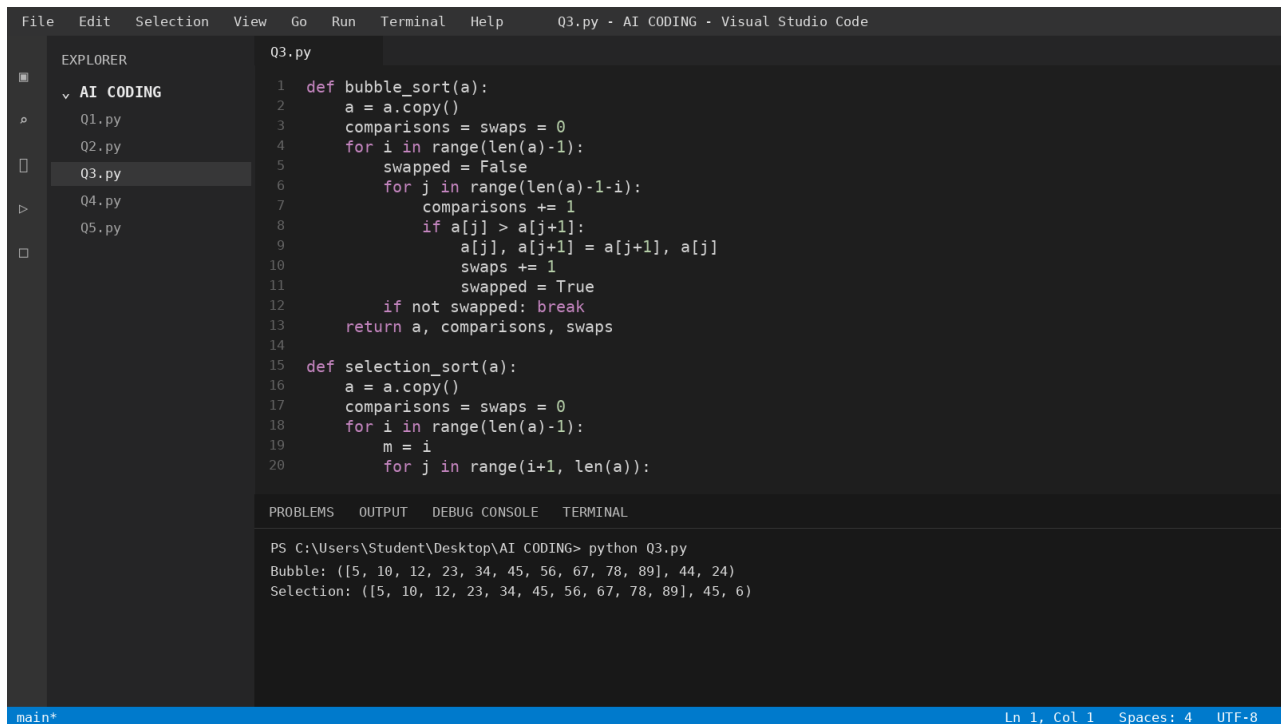
```
# TODO:
```

```
# Sort using Bubble Sort, Selection Sort and Insertion Sort.
```

```
# Compare their performance.
```

Student Tasks:

- Implement all three sorting algorithms.
- Use the same input for each algorithm.
- Count comparisons and swaps/shifts.
- Use AI to explain strengths and weaknesses.
- Ask AI which algorithm suits nearly sorted data.
- Compare experimental results with theoretical complexity.



```
File Edit Selection View Go Run Terminal Help Q3.py - AI CODING - Visual Studio Code

EXPLORER
  AI CODING
    Q1.py
    Q2.py
    Q3.py
    Q4.py
    Q5.py

Q3.py
1  def bubble_sort(a):
2      a = a.copy()
3      comparisons = swaps = 0
4      for i in range(len(a)-1):
5          swapped = False
6          for j in range(len(a)-1-i):
7              comparisons += 1
8              if a[j] > a[j+1]:
9                  a[j], a[j+1] = a[j+1], a[j]
10                 swaps += 1
11                 swapped = True
12             if not swapped: break
13         return a, comparisons, swaps
14
15     def selection_sort(a):
16         a = a.copy()
17         comparisons = swaps = 0
18         for i in range(len(a)-1):
19             m = i
20             for j in range(i+1, len(a)):

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\Student\Desktop\AI CODING> python Q3.py
Bubble: ([5, 10, 12, 23, 34, 45, 56, 67, 78, 89], 44, 24)
Selection: ([5, 10, 12, 23, 34, 45, 56, 67, 78, 89], 45, 6)
```

All three produce: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89].

Experimental comparison: Bubble Sort = 44 comparisons, 24 swaps; Selection Sort = 45 comparisons, 6 swaps; Insertion Sort = 31 comparisons, 24 shifts.

Strengths and weaknesses:

Bubble Sort: simple, but normally $O(n^2)$ and may perform many swaps. Early exit helps nearly sorted data.

Selection Sort: performs relatively few swaps, but still makes $O(n^2)$ comparisons.

Insertion Sort: very suitable for small or nearly sorted data because only misplaced elements need to move.

Nearly sorted data: Insertion Sort is the best choice among these three.

Theoretical complexity: Optimised Bubble Sort best $O(n)$, average/worst $O(n^2)$; Selection Sort best/average/worst $O(n^2)$; Insertion Sort best $O(n)$, average/worst $O(n^2)$.

Q4 – Optimising a Student Search and Sort Program

Problem / Starter Code:

```
students = [  
    ("Arun", 82),  
    ("Bala", 65),  
    ("Chitra", 91),
```

```
("Divya", 74),  
("Esha", 88)  
]
```

TODO:

Find the highest mark.

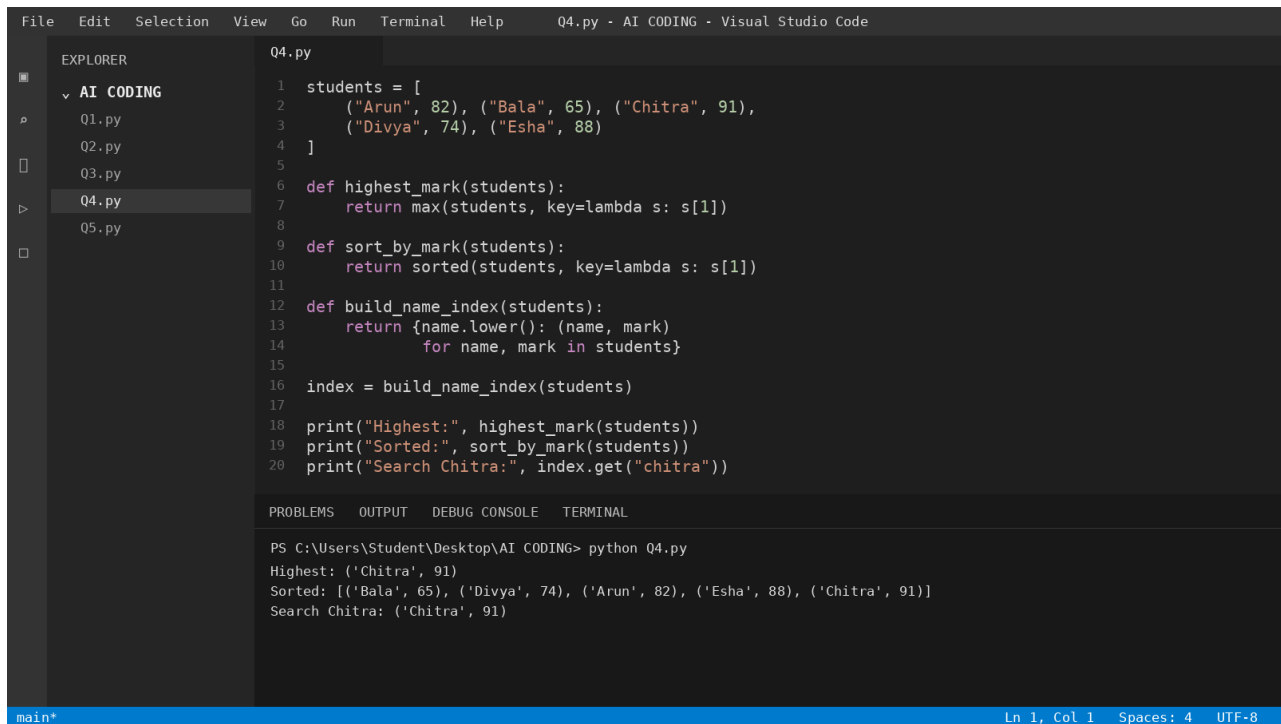
Sort students by mark.

Search for a student by name.

Improve the solution using AI.

Student Tasks:

- Write functions for highest mark, sorting and name search.
- Create a straightforward solution first.
- Use AI to identify unnecessary operations.
- Ask AI for a more efficient approach.
- Implement only suggestions you understand.
- Compare original and improved time complexity.



```
File Edit Selection View Go Run Terminal Help Q4.py - AI CODING - Visual Studio Code

EXPLORER
v AI CODING
  Q1.py
  Q2.py
  Q3.py
  Q4.py
  Q5.py

Q4.py
1 students = [
2     ("Arun", 82), ("Bala", 65), ("Chitra", 91),
3     ("Divya", 74), ("Esha", 88)
4 ]
5
6 def highest_mark(students):
7     return max(students, key=lambda s: s[1])
8
9 def sort_by_mark(students):
10    return sorted(students, key=lambda s: s[1])
11
12 def build_name_index(students):
13    return {name.lower(): (name, mark)
14            for name, mark in students}
15
16 index = build_name_index(students)
17
18 print("Highest:", highest_mark(students))
19 print("Sorted:", sort_by_mark(students))
20 print("Search Chitra:", index.get("chitra"))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\Student\Desktop\AI CODING> python Q4.py
Highest: ('Chitra', 91)
Sorted: [('Bala', 65), ('Divya', 74), ('Arun', 82), ('Esha', 88), ('Chitra', 91)]
Search Chitra: ('Chitra', 91)

main* Ln 1, Col 1 Spaces: 4 UTF-8
```

Output: Highest = Chitra, 91. Sorted by mark = Bala–65, Divya–74, Arun–82, Esha–88, Chitra–91. Chitra search = (Chitra, 91).

AI-suggested improvement:

If the program performs many name searches, scanning the complete list for every search repeats work. Build a name index once and reuse it.

Complexity comparison: highest mark remains $O(n)$; sorting remains $O(n \log n)$ using Python's sorted; original name search is $O(n)$ per search, while indexed name search is $O(1)$ average after $O(n)$ index construction.

The index uses $O(n)$ extra space. It is useful when many name searches are performed; for only one tiny search, building an index may not be necessary.

Q5 – Optimisation Challenge: Best Combination

Problem / Starter Code:

```
items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
```

("Headphones", 5000)

]

budget = 50000

TODO:

Find a combination of items whose total price

is as high as possible without exceeding the budget.

Student Tasks:

- Design a solution to maximise total value without exceeding the budget.
- Start with a simple approach and explain it.
- Use AI to compare Brute Force, Greedy and Dynamic Programming.
- Determine which approach guarantees an optimal solution for this formulation.
- Implement the selected solution.
- Test at least three budgets.
- Explain time and space complexity and possible optimisation for larger inputs.

```
File Edit Selection View Go Run Terminal Help Q5.py - AI CODING - Visual Studio Code

EXPLORER
  AI CODING
    Q1.py
    Q2.py
    Q3.py
    Q4.py
    Q5.py

Q5.py
1  def knapsack_dp(items, budget):
2      dp = [0] * (budget + 1)
3      chosen = [[] for _ in range(budget + 1)]
4
5      for name, price in items:
6          for b in range(budget, price - 1, -1):
7              candidate = dp[b-price] + price
8              if candidate > dp[b]:
9                  dp[b] = candidate
10                 chosen[b] = chosen[b-price] + [name]
11     return chosen[budget], dp[budget]
12
13     items = [
14         ("Laptop", 60000), ("Phone", 30000),
15         ("Tablet", 20000), ("Watch", 10000),
16         ("Headphones", 5000)
17     ]
18
19     for budget in [50000, 65000, 100000]:
20         print(budget, "->", knapsack_dp(items, budget))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\Student\Desktop\AI CODING> python Q5.py
50000 -> (['Phone', 'Tablet'], 50000)
65000 -> (['Laptop', 'Headphones'], 65000)
100000 -> (['Laptop', 'Phone', 'Watch'], 100000)

main* Ln 1, Col 1 Spaces: 4 UTF-8
```

For budget ₹50,000, the best combination is Phone + Tablet = ₹50,000.

Greedy vs Dynamic Programming:

Greedy makes the best local choice at each step, but for the general 0/1 item-selection formulation it does not guarantee the global optimum.

0/1 Dynamic Programming guarantees the optimal answer for this formulation because it considers the best achievable total for smaller budgets while ensuring each item is used at most once.

Test results:

Budget ₹50,000 → Phone + Tablet → ₹50,000.

Budget ₹65,000 → Laptop + Headphones → ₹65,000.

Budget ₹100,000 → Laptop + Phone + Watch → ₹100,000.

Complexity: Brute Force checks all subsets, $O(2^n)$ time. 0/1 Dynamic Programming is $O(nB)$ time and $O(B)$ extra space for budget B . For larger inputs, DP can be expensive when B is very large; reducing the state space or using a problem-specific optimisation may help.

Final choice: Dynamic Programming is selected because the assignment asks for a method that guarantees the optimal combination for the stated 0/1 formulation.